

Explication Ultra-Détaillée

Projet Deep Learning — EMSI Casablanca

2025–2026

Guide complet section par section — 3 Parties

Partie I : MLP (PyTorch) | Partie II : CNN | Partie III : RNN/LSTM/GRU/Seq2Seq

Dataset I : Breast Cancer Wisconsin | Dataset II : Fashion-MNIST | Dataset III : EN→FR

Partie	Thème	Points	Concepts clés
I	MLP et ingénierie PyTorch	30 pts	nn.Module, init, BCELoss, state_dict
II	CNN et vision par ordinateur	35 pts	corr2d, pooling, LeNet, feature maps
III	RNN, LSTM, GRU, Seq2Seq	35 pts	BPTT, clip_grad, Teacher Forcing, BLEU
★	Question transversale finale	Bonus	Comparaison architectures, biais inductif

PARTIE I — MLP et ingénierie PyTorch

Section I.1 — Concepts fondamentaux PyTorch

Qu'est-ce qu'on fait ici ?

Avant d'écrire une seule ligne de modèle, on pose les briques conceptuelles de PyTorch : **nn.Module**, **named_parameters()** et **state_dict()**. Ces trois éléments forment le squelette de tout modèle PyTorch.

nn.Module — La brique fondamentale

```
import torch.nn as nn

class MonModele(nn.Module):
    def __init__(self):
        super().__init__() # obligatoire : initialise la machinerie interne
        self.couche = nn.Linear(10, 5) # couche avec poids

    def forward(self, x): # propagation avant
        return self.couche(x)
```

nn.Module est la classe parente de tous les modèles PyTorch. Elle gère automatiquement : l'enregistrement des paramètres (poids), le déplacement vers GPU (`.to(device)`), le mode eval/train, et la sauvegarde. **super().__init__()** est obligatoire pour activer cette machinerie.

forward() définit le chemin que suivent les données. On ne l'appelle jamais directement — on appelle le modèle comme une fonction : $y = model(x)$, ce qui déclenche automatiquement `model.forward(x)`.

named_parameters() — Inspecter les poids

```
for name, param in model.named_parameters():
    print(f'{name}: forme={param.shape}, grad={param.requires_grad}')

# Exemple de sortie :
# couches.0.weight : forme=torch.Size([128, 30]), grad=True
# couches.0.bias : forme=torch.Size([128]), grad=True
```

Méthode	Retourne	Utilité
<code>model.parameters()</code>	Itérateur sur les tenseurs de poids	Passé à l'optimiseur
<code>model.named_parameters()</code>	Paires (nom, tenseur)	Debug, inspection
<code>model.state_dict()</code>	Dict {nom: tenseur} de tous les paramètres	Sauvegarde/chargement

Propagation avant et rétropropagation

```
# Propagation avant
y_pred = model(X_batch) # forward : calcule la prédiction
loss = criterion(y_pred, y) # calcule la perte
```

```
# Rétropropagation
optimizer.zero_grad() # efface les gradients précédents
loss.backward() # calcule les gradients (autograd)
optimizer.step() # met à jour les poids
```

PyTorch construit un **graphe de calcul dynamique** à chaque forward. `loss.backward()` remonte ce graphe pour calculer le gradient de la perte par rapport à chaque paramètre (règle de la chaîne). `zero_grad()` est obligatoire car PyTorch ACCUMULE les gradients par défaut.

Ce qu'on dit à l'oral

PyTorch est organisé autour de `nn.Module`. Tout modèle hérite de cette classe qui gère automatiquement les paramètres, le GPU, et la sauvegarde. La méthode `forward` définit les calculs. La rétropropagation est automatique grâce au système `autograd` de PyTorch — il suffit d'appeler `loss.backward()` et PyTorch calcule tous les gradients seul.

Section I.2 — Chargement et préparation — Breast Cancer Wisconsin

Qu'est-ce qu'on fait ici ?

On charge le dataset Breast Cancer Wisconsin (sklearn), on l'explore, puis on le prépare pour PyTorch : normalisation, split 70/15/15, conversion en Tenseurs, DataLoaders.

Le dataset Breast Cancer Wisconsin

Propriété	Valeur
Source	sklearn.datasets.load_breast_cancer()
Observations	569 tumeurs
Features	30 mesures nucléaires (rayon, texture, périmètre, aire, ...)
Cible	0 = Maligne (cancéreuse) 1 = Bénigne (non cancéreuse)
Tâche	Classification binaire supervisée
Déséquilibre	357 bénignes (63%) vs 212 malignes (37%) — léger déséquilibre

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
X_raw, y_raw = data.data, data.target # X: (569,30), y: (569,)

# Normalisation sklearn
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw) # moyenne=0, std=1

# Split 70% train / 15% val / 15% test
X_tr_v, X_te, y_tr_v, y_te = train_test_split(X_scaled, y_raw,
test_size=0.15, random_state=42, stratify=y_raw)
X_tr, X_val, y_tr, y_val = train_test_split(X_tr_v, y_tr_v,
test_size=0.176, random_state=42, stratify=y_tr_v)
```

stratify=y_raw garantit que la proportion 63%/37% est conservée dans chaque split. Sans ça, un split aléatoire pourrait mettre 100% des cas malins dans le test par malchance.

Conversion en Tenseurs PyTorch et DataLoader

```
# Conversion numpy → tenseur PyTorch
X_tr_t = torch.FloatTensor(X_tr)
y_tr_t = torch.FloatTensor(y_tr).unsqueeze(1) # (N,) -> (N,1)

# DataLoader : gère les mini-batches automatiquement
train_ds = TensorDataset(X_tr_t, y_tr_t)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
```

Pourquoi unsqueeze(1) ? BCELoss attend des tenseurs de forme (N,1) pour les labels. Si on garde (N,), il y a une erreur de dimension. unsqueeze(1) ajoute une dimension : (N,) devient (N,1).

Ce qu'on dit à l'oral

On utilise le dataset Breast Cancer Wisconsin : 569 tumeurs décrites par 30 mesures nucléaires. La tâche est de classer chaque tumeur comme maligne ou bénigne. On normalise d'abord avec `StandardScaler`, on fait un split 70/15/15 stratifié pour respecter l'équilibre des classes, puis on convertit en tenseurs PyTorch avec `DataLoader` pour les mini-batches.

Section I.3 — Deux versions d'un MLP

Qu'est-ce qu'on fait ici ?

On implémente le même MLP de deux façons différentes : **nn.Sequential** (compact, pour architectures simples) et **classe nn.Module** (flexible, pour architectures complexes). Comprendre les deux est une compétence fondamentale.

Version 1 — nn.Sequential

```
mlp_seq = nn.Sequential(
    nn.Linear(30, 128), # couche d'entree : 30 features -> 128 neurones
    nn.ReLU(), # activation non-lineaire
    nn.Dropout(0.3), # regularisation : eteint 30% des neurones aleatoirement
    nn.Linear(128, 64), # couche cachee : 128 -> 64
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(64, 1), # sortie : 1 neurone (classification binaire)
    nn.Sigmoid() # ecrase la sortie entre 0 et 1 (probabilite)
).to(device)
```

nn.Sequential enchaîne les couches dans l'ordre. L'entrée passe dans la couche 1, la sortie va dans la couche 2, etc. C'est pratique mais limité : pas de branches, pas d'opérations conditionnelles.

Version 2 — Classe personnalisée nn.Module

```
class MLPCustom(nn.Module):
    def __init__(self, input_dim=30, hidden=[128,64], output_dim=1, dropout=0.3):
        super().__init__()
        # Créer les couches dynamiquement selon la liste hidden
        self.couches = nn.ModuleList()
        dims = [input_dim] + hidden
        for i in range(len(hidden)):
            self.couches.append(nn.Linear(dims[i], dims[i+1]))
        self.sortie = nn.Linear(hidden[-1], output_dim)
        self.dropout = nn.Dropout(dropout)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        for couche in self.couches:
            x = F.relu(couche(x))
        x = self.dropout(x)
        return self.sigmoid(self.sortie(x))
```

Concept	Explication
nn.ModuleList	Liste de couches enregistrée par PyTorch — contrairement à une list Python classique
dims = [input_dim] + hidden	Si hidden=[128,64] et input=30 → dims=[30,128,64]
boucle for	Crée Linear(30,128) puis Linear(128,64) automatiquement
F.relu()	ReLU appliquée fonctionnellement (pas un module enregistré)

Pourquoi `nn.ModuleList` et pas une `list` Python ? Si on utilise `list()` standard, PyTorch ne reconnaît pas les couches comme paramètres du modèle → `model.parameters()` retourne vide → rien ne s'entraîne.

Ce qu'on dit à l'oral

`nn.Sequential` convient pour des architectures simples en ligne droite. La classe personnalisée est nécessaire dès qu'on veut de la flexibilité : architectures avec branches, connexions résiduelles, logique conditionnelle. La version classe permet aussi de générer l'architecture dynamiquement depuis une liste de dimensions — très pratique pour les expériences.

Section I.4 — Inspection des paramètres

Qu'est-ce qu'on fait ici ?

On utilise `named_parameters()` pour explorer la structure interne du modèle : noms des couches, formes des matrices de poids, et nombre total de paramètres entraînables.

```
total = 0
for name, param in mlp_custom.named_parameters():
    n = param.numel() # nombre d'elements dans le tenseur
    print(f'{name:30s} | forme={str(param.shape):20s} | params={n:6d}')
    total += n
print(f'Total parametres entraînables : {total:,}')
```

Calcul du nombre de paramètres d'un MLP 30→128→64→1 :

Couche	Matrice de poids	Biais	Total
Linear(30, 128)	30×128 = 3 840	128	3 968
Linear(128, 64)	128×64 = 8 192	64	8 256
Linear(64, 1)	64×1 = 64	1	65
TOTAL			12 289 paramètres

Formule : une couche `Linear(n_in, n_out)` a $n_{in} \times n_{out}$ poids + n_{out} biais = $n_{out} \times (n_{in} + 1)$ paramètres.

Ce qu'on dit à l'oral

`named_parameters()` nous permet de vérifier la structure du modèle et de compter les paramètres. Notre MLP a ~12 000 paramètres. C'est peu comparé aux CNN (millions) ou aux Transformers (milliards). Connaître ce nombre aide à estimer la mémoire nécessaire et le risque d'overfitting par rapport à la taille du dataset.

Section I.5 — Stratégies d'initialisation des poids

Qu'est-ce qu'on fait ici ?

L'initialisation des poids détermine la vitesse de convergence et la qualité finale du modèle. On compare 3 stratégies : **Gaussienne**, **Constante**, et **Xavier Uniforme**.

Stratégie	Formule	Problème potentiel	Quand utiliser
Gaussienne $N(0, 0.01)$	poids $\sim N(0, 0.01)$	Gradients très petits au départ → convergence lente	Réseau peu profond, baseline
Constante (1.0)	tous poids = 1.0	BRISURE DE SYMÉTRIE : tous neurones d'une couche identiques → n'apprennent pas	NE PAS UTILISER — démonstration pédagogique
Xavier Uniforme	poids $\sim U[-\sqrt{6/(n_{in}+n_{out})}, +\sqrt{6/(n_{in}+n_{out})}]$	Aucun si activations ReLU/Tanh	Réseaux profonds — recommandé

```
def init_gaussienne(module):
    if isinstance(module, nn.Linear):
        nn.init.normal_(module.weight, mean=0.0, std=0.01)
        nn.init.zeros_(module.bias)

def init_constant(module):
    if isinstance(module, nn.Linear):
        nn.init.constant_(module.weight, 1.0) # probleme symetrie!
        nn.init.zeros_(module.bias)

def init_xavier(module):
    if isinstance(module, nn.Linear):
        nn.init.xavier_uniform_(module.weight)
        nn.init.zeros_(module.bias)

# Application : model.apply() appelle la fonction sur chaque sous-module
model.apply(init_xavier)
```

Pourquoi Xavier est la meilleure initialisation ?

Intuition : Xavier calcule la plage d'initialisation en fonction du nombre d'entrées (f_{in}) et de sorties (f_{out}) de chaque couche. Objectif : que la variance du signal reste stable d'une couche à l'autre. Sans ça, les signaux exploseraient ou s'évanouiraient dans les réseaux profonds.

Le problème de symétrie avec l'init constante : Si tous les poids d'une couche ont la même valeur, tous les neurones reçoivent le même gradient et se mettent à jour identiquement. Ils restent identiques pour toujours → la couche équivaut à 1 seul neurone.

Ce qu'on dit à l'oral

L'initialisation constante casse l'apprentissage car tous les neurones restent identiques — c'est le problème de symétrie. La gaussienne fonctionne mais peut être lente. Xavier est la meilleure car elle maintient la variance du signal stable en tenant compte du nombre d'entrées et de sorties de chaque couche.

Section I.6 — Boucle d'entraînement

Qu'est-ce qu'on fait ici ?

On écrit la boucle d'entraînement complète : forward, calcul de la perte, backward, mise à jour des poids. C'est le coeur de tout entraînement PyTorch.

```
def train_model(model, train_loader, val_loader, epochs=50, lr=1e-3):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.BCELoss() # Binary Cross-Entropy
    history = {'train_loss':[], 'val_loss':[], 'val_acc':[]}

    for epoch in range(epochs):
        model.train() # mode entraînement (Dropout actif)
        for X_b, y_b in train_loader:
            X_b, y_b = X_b.to(device), y_b.to(device)
            optimizer.zero_grad() # reinitialiser gradients
            y_pred = model(X_b) # forward
            loss = criterion(y_pred, y_b) # perte
            loss.backward() # retropropagation
            optimizer.step() # mise à jour poids

        model.eval() # mode evaluation (Dropout désactive)
        with torch.no_grad(): # pas de calcul de gradient
            y_val_pred = model(X_val_t)
            val_loss = criterion(y_val_pred, y_val_t).item()
            val_acc = ((y_val_pred > 0.5) == y_val_t).float().mean()
```

Élément	Rôle	Détail
Adam	Optimiseur adaptatif	Adapte le lr pour chaque paramètre — plus rapide que SGD
BCELoss	Perte classification binaire	$-(y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}))$ — convient à Sigmoid en sortie
model.train()	Active Dropout et BatchNorm	Pendant l'entraînement : Dropout éteint des neurones
model.eval()	Désactive Dropout	Pendant l'évaluation : tous les neurones actifs
torch.no_grad()	Pas de graphe de calcul	Économise mémoire/calcul pendant l'évaluation
zero_grad()	Réinitialise les gradients	PyTorch accumule : sans zero_grad, gradients s'empilent

Ce qu'on dit à l'oral

La boucle PyTorch suit toujours le même schéma : zero_grad, forward, calcul de la perte, backward, step. BCELoss est utilisée car notre sortie est un Sigmoid qui produit une probabilité entre 0 et 1. model.train() active le Dropout pendant l'entraînement pour régulariser, model.eval() le désactive pendant l'évaluation pour avoir des prédictions stables.

Section I.7 — Sauvegarde et rechargement du modèle

Qu'est-ce qu'on fait ici ?

On sauvegarde le meilleur modèle (celui avec la meilleure `val_loss`) et on le recharge. C'est fondamental pour ne pas perdre l'entraînement et pour déployer un modèle en production.

```
# Sauvegarde du state_dict (poids uniquement – RECOMMANDE)  
torch.save(best_state, 'best_mlp_p1.pth')  
  
# Rechargement  
model_loaded = MLPCustom().to(device) # recree l'architecture  
model_loaded.load_state_dict(  
    torch.load('best_mlp_p1.pth', map_location=device)  
)  
model_loaded.eval() # passer en mode evaluation
```

Méthode	Sauvegarde quoi ?	Avantage	Inconvénient
<code>torch.save(model.state_dict())</code>	Poids uniquement (dict)	Léger, portable, recommandé	Il faut recréer l'architecture manuellement
<code>torch.save(model)</code>	Modèle complet (pickle)	Rechargement facile sans redéfinir	Lié à la structure de classe Python — fragile

`map_location=device` est crucial : si le modèle a été sauvegardé sur GPU mais qu'on recharge sur CPU (ou vice-versa), PyTorch gère automatiquement la conversion de device.

Ce qu'on dit à l'oral

On sauvegarde le `state_dict` — le dictionnaire des poids — plutôt que le modèle entier. C'est plus portable. Pour recharger, on recrée d'abord l'architecture puis on charge les poids avec `load_state_dict`. `map_location` permet de charger un modèle entraîné sur GPU sur une machine sans GPU.

Section I.8 — Gestion du device CPU/GPU

Qu'est-ce qu'on fait ici ?

On vérifie si un GPU est disponible et on déplace modèle et données vers le bon device. C'est ce qui permet au même code de tourner sur laptop (CPU) et sur serveur de calcul (GPU).

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device utilise : {device}')

# Modele -> device
model = MLPCustom().to(device)

# Donnees -> device a chaque batch
for X_b, y_b in train_loader:
    X_b, y_b = X_b.to(device), y_b.to(device)
    ...

# Verification
print(next(model.parameters()).device) # cuda:0 ou cpu
```

Pourquoi déplacer les données à chaque batch ? Déplacer tout le dataset d'un coup en GPU occupe la mémoire GPU inutilement. On déplace uniquement le mini-batch courant au moment de l'utiliser.

Ce qu'on dit à l'oral

`device = 'cuda' if torch.cuda.is_available() else 'cpu'` est la ligne universelle PyTorch. Le même code tourne sur CPU ou GPU sans modification. On déplace le modèle avec `.to(device)` une seule fois, et les données à chaque batch.

Section I.9 — Évaluation complète sur le test set

Qu'est-ce qu'on fait ici ?

On évalue le modèle rechargé sur le test set (jamais vu pendant l'entraînement) avec 5 métriques : accuracy, precision, recall, F1-score et matrice de confusion.

Métrique	Formule	Interprétation médicale	Meilleur
Accuracy	$(VP+VN) / \text{Total}$	Taux de bonnes classifications	Proche 1
Precision	$VP / (VP+FP)$	Parmi les détectés malins, combien le sont vraiment ?	Proche 1
Recall	$VP / (VP+FN)$	Parmi les vrais malins, combien détectés ? (sensibilité)	Proche 1 — CRITIQUE
F1-Score	$2 \times (P \times R) / (P + R)$	Moyenne harmonique Precision/Recall — équilibre	Proche 1
AUC-ROC	Aire sous la courbe ROC	Capacité discriminante globale	Proche 1

Pourquoi le Recall est critique en médecine ? Un faux négatif (tumeur maligne classée bénigne) est catastrophique — le patient ne sera pas traité. Un faux positif (bénigne classée maligne) mène à des examens supplémentaires, inconfortable mais pas mortel. On préfère un Recall élevé même si la Precision baisse légèrement.

```
model_loaded.eval()
with torch.no_grad():
    y_prob = model_loaded(X_te_t).cpu().numpy().ravel()
    y_pred_class = (y_prob > 0.5).astype(int)

from sklearn.metrics import classification_report, confusion_matrix
print(classification_report(y_te, y_pred_class,
    target_names=['Maligne', 'Benigne']))
cm = confusion_matrix(y_te, y_pred_class)
```

Ce qu'on dit à l'oral

On évalue sur le test set avec les 5 métriques standard de classification. En oncologie, le Recall (sensibilité) est la métrique la plus importante car manquer un cancer est beaucoup plus grave que faire une fausse alarme. Notre MLP atteint typiquement F1~0.97 sur ce dataset.

Section I.10 — Question de synthèse — Partie I

Question : Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification de données tabulaires médicales ?

Arguments POUR le MLP sur données tabulaires :

- Les 30 features sont indépendantes et bien définies — pas de structure spatiale ou temporelle
- Le MLP peut capturer les non-linéarités que la régression logistique ne peut pas
- Avec seulement 569 observations, un modèle léger (~12k params) est adapté
- Dropout et xavier offrent une bonne régularisation

Limites du MLP sur ce dataset :

- 569 exemples est très peu pour du deep learning — un SVM ou RF peut faire aussi bien
- Le MLP ignore l'ordre ou les corrélations entre features — il les traite de façon indépendante
- Pas d'interprétabilité native : impossible de savoir quelle mesure influence la décision

Conclusion : Un MLP est pertinent pour des données tabulaires médicales à condition d'avoir suffisamment de données (>1000 exemples idéalement). Sur ce dataset, il rivalise avec les méthodes classiques comme SVM ou Random Forest qui sont souvent préférées pour leur interprétabilité en médecine.

PARTIE II — CNN et vision par ordinateur

Section II.1 — Pourquoi le MLP est inadapté aux images

Qu'est-ce qu'on fait ici ?

Avant d'introduire les CNN, on justifie leur nécessité en montrant les limites fondamentales du MLP pour traiter des images.

Problème	MLP sur images	CNN
Dimensionnalité	Image $28 \times 28 = 784$ pixels $\rightarrow$ vecteur 784D. Image $224 \times 224 = 150k$ entrées!	Fenêtre glissante locale — traite des patches
Paramètres	Couche $784 \rightarrow 512 = 401\,408$ poids juste pour une couche	Partage de poids : même filtre sur toute l'image
Invariance	Si on décale l'objet de 2 pixels $\rightarrow$ prédiction différente	Convolution = invariante à la translation
Structure spatiale	Aplatit l'image $\rightarrow$ détruit les voisinages 2D	Traite les pixels dans leur contexte spatial

Exemple concret : un MLP sur Fashion-MNIST 28×28 avec une couche cachée de 512 neurones a $784 \times 512 + 512 = 401\,920$ paramètres. Un LeNet sur les mêmes images a seulement $\sim 60\,000$ paramètres grâce au partage de poids, et est beaucoup plus précis.

Ce qu'on dit à l'oral

Le MLP aplatit l'image en vecteur, ce qui détruit l'information spatiale. Il ne sait pas que le pixel (5,5) est voisin du pixel (5,6). Il est aussi sensible aux translations : décaler un objet de 2 pixels donne une prédiction différente. Le CNN résout ces problèmes avec la convolution locale et le partage de poids.

Section II.2 — Implémentation manuelle de corr2d

Qu'est-ce qu'on fait ici ?

On implémente la corrélation croisée 2D à la main pour comprendre ce que fait réellement `nn.Conv2d` sous le capot.

```
def corr2d(X, K):
    """Correlation croisee 2D : X=image (H,W), K=noyau (kh,kw)"""
    h, w = K.shape
    # Taille de sortie : (H-kh+1) x (W-kw+1)
    Y = torch.zeros(X.shape[0] - h + 1, X.shape[1] - w + 1)
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            region = X[i:i+h, j:j+w] # patch de taille kh x kw
            Y[i, j] = (region * K).sum() # produit element-wise + somme
    return Y
```

Interprétation : on fait glisser le noyau `K` sur l'image `X`. Pour chaque position, on multiplie chaque pixel du patch par le poids correspondant du noyau, puis on somme. Si `K` est un détecteur de contour vertical, la sortie sera élevée là où il y a un contour vertical.

Taille de sortie sans padding :

Sortie = $(H - k_h + 1) \times (W - k_w + 1)$
Exemple : image 6×8, noyau 3×3 → sortie $(6-3+1) \times (8-3+1) = 4 \times 6$

Exemple — Détection de contours :

```
# Noyau detecteur de contour vertical
K = torch.tensor([[1., -1.],
                  [1., -1.]])

# Sur une image avec une bande noire au centre :
# +1 la ou il y a une transition blanc->noir
# -1 la ou il y a une transition noir->blanc
# 0 la ou pas de contour
```

Ce qu'on dit à l'oral

La corrélation croisée 2D fait glisser un noyau sur l'image. Pour chaque position, on multiplie les pixels du patch par les poids du noyau et on somme. C'est ce que fait `Conv2d` en PyTorch, mais de façon optimisée. L'implémentation manuelle permet de comprendre exactement le mécanisme avant d'utiliser la version boîte noire.

Section II.3 — Implémentation manuelle du pooling

Qu'est-ce qu'on fait ici ?

On implémente le max-pooling et l'average-pooling à la main pour comprendre leur rôle de réduction de dimension et d'invariance locale.

```
def max_pool2d_manual(X, pool_size=(2,2), stride=None):
    ph, pw = pool_size
    sh, sw = stride if stride else pool_size # stride=pool par défaut
    H_out = (X.shape[0] - ph) // sh + 1
    W_out = (X.shape[1] - pw) // sw + 1
    Y = torch.zeros(H_out, W_out)
    for i in range(H_out):
        for j in range(W_out):
            region = X[i*sh:i*sh+ph, j*sw:j*sw+pw]
            Y[i, j] = region.max() # valeur maximale du patch
    return Y
```

Pooling	Opération	Avantage	Inconvénient
Max-Pooling	max() sur chaque patch	Capture la présence d'une feature (robuste)	Perd la localisation exacte
Avg-Pooling	mean() sur chaque patch	Lisse, plus stable	Dilue les features fortes
Global Avg	mean() sur toute la carte	Réduit à 1 valeur par filtre — fin de réseau	Perd toute info spatiale

Rôle fondamental du pooling : réduire la taille spatiale ($28 \times 28 \rightarrow 14 \times 14$ avec pool 2×2) ce qui réduit le nombre de paramètres, donne une invariance aux petites translations, et augmente le champ récepteur des couches suivantes.

Ce qu'on dit à l'oral

Le pooling réduit la taille spatiale tout en conservant l'information essentielle. Max-pooling dit "est-ce que cette feature est présente quelque part dans ce patch ?". Average-pooling lisse la réponse. Après un pooling 2×2 , une carte 28×28 devient 14×14 — on divise le nombre de calculs par 4.

Section II.4 — Calculs de padding et stride

Qu'est-ce qu'on fait ici ?

On présente les formules de calcul des tailles de sortie avec padding et stride, et on vérifie expérimentalement avec PyTorch.

Formule générale :

$$H_{out} = \text{floor}((H_{in} + 2 \times p_h - k_h) / s_h) + 1$$

$$W_{out} = \text{floor}((W_{in} + 2 \times p_w - k_w) / s_w) + 1$$

Paramètre	Rôle	Valeur typique
padding (p)	Ajoute des zéros autour de l'image	$p=k/2$ pour conserver la taille ('same')
stride (s)	Pas du déplacement du noyau	$s=1$ normal, $s=2$ divise la taille par 2
kernel (k)	Taille du noyau de convolution	3×3 (standard), 5×5 (LeNet), 1×1 (projection)

```
# Exemple complet : image 28x28, Conv2d(1,6,5,padding=2), MaxPool2d(2,2)
# Après Conv : floor((28 + 2*2 - 5)/1) + 1 = floor(27/1)+1 = 28 -> 28x28
# Après Pool : floor((28 - 2)/2) + 1 = 14 -> 14x14

def output_size(H, W, k_h, k_w, p_h=0, p_w=0, s_h=1, s_w=1):
    H_out = (H + 2*p_h - k_h) // s_h + 1
    W_out = (W + 2*p_w - k_w) // s_w + 1
    return H_out, W_out
```

Convolution 1×1 : un noyau 1×1 ne capte pas les relations spatiales mais projette les canaux : Conv2d(64, 32, kernel_size=1) réduit 64 canaux à 32 avec un coût minimal. Utilisé dans GoogLeNet/Inception pour réduire la dimension avant des convolutions coûteuses.

Ce qu'on dit à l'oral

padding=2 avec noyau 5×5 conserve la taille spatiale de l'image. stride=2 divise la résolution par 2 — c'est une alternative au pooling. La convolution 1×1 est contre-intuitive : elle ne regarde qu'un pixel mais combine ses canaux — c'est une projection de dimension.

Section II.5 — Chargement Fashion-MNIST

Qu'est-ce qu'on fait ici ?

On charge Fashion-MNIST via torchvision, on normalise, et on crée les DataLoaders.

Propriété	Valeur
Dataset	Fashion-MNIST (Zalando Research)
Images	70 000 images 28×28 pixels en niveaux de gris
Classes	10 : T-shirt, Pantalon, Pull, Robe, Manteau, Sandale, Chemise, Sneaker, Sac, Bottine
Train / Test	60 000 / 10 000
Normalisation	mean=0.2860, std=0.3530 (calculé sur le train set)

```
transform = transforms.Compose([
    transforms.ToTensor(), # PIL -> [0,1] float tensor
    transforms.Normalize((0.2860,), (0.3530,)) # normalisation canal gris
])

train_fmnist = torchvision.datasets.FashionMNIST(
    root='./data', train=True, download=True, transform=transform)
train_loader_fmnist = DataLoader(train_fmnist, batch_size=64, shuffle=True)
```

Ce qu'on dit à l'oral

Fashion-MNIST remplace MNIST classique : même structure (28×28, 10 classes) mais plus difficile — les chiffres manuscrits sont trop simples. torchvision gère le téléchargement et le prétraitement automatiquement. La normalisation centre les pixels : valeur typique ~0 au lieu de 0.28.

Section II.6 — MLP baseline sur Fashion-MNIST

Qu'est-ce qu'on fait ici ?

On entraîne un MLP simple sur Fashion-MNIST comme point de référence pour montrer l'amélioration apportée par le CNN.

```
class MLPBaseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(), # (B,1,28,28) -> (B,784)
            nn.Linear(784, 256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128, 10) # 10 classes, pas de Softmax (inclus dans loss)
        )
```

Pourquoi pas de Softmax en sortie ? nn.CrossEntropyLoss combine LogSoftmax + NLLLoss. Ajouter un Softmax avant et un CrossEntropyLoss après ferait deux Softmax → erreur de calcul. La convention PyTorch : logits bruts + CrossEntropyLoss.

Le MLP baseline atteint typiquement **87-89% d'accuracy** sur Fashion-MNIST. Il servira de référence pour montrer que LeNet fait mieux (~91-93%).

Ce qu'on dit à l'oral

Le MLP baseline aplatit l'image avec nn.Flatten() avant de la traiter. Il atteint ~88% d'accuracy. nn.CrossEntropyLoss inclut déjà le Softmax donc on ne l'ajoute pas en sortie du modèle. Ce baseline nous donnera la comparaison avec LeNet.

Section II.7 — Architecture LeNet

Qu'est-ce qu'on fait ici ?

On implémente LeNet-5 (LeCun 1998), l'architecture CNN fondatrice, adaptée à Fashion-MNIST (28×28 au lieu de 32×32).

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        # Bloc convolutif 1 : (1,28,28) -> (6,14,14)
        self.conv1 = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5, padding=2), # (1,28,28)->(6,28,28)
            nn.ReLU(),
            nn.MaxPool2d(2, 2) # (6,28,28)->(6,14,14)
        )
        # Bloc convolutif 2 : (6,14,14) -> (16,5,5)
        self.conv2 = nn.Sequential(
            nn.Conv2d(6, 16, kernel_size=5), # (6,14,14)->(16,10,10)
            nn.ReLU(),
            nn.MaxPool2d(2, 2) # (16,10,10)->(16,5,5)
        )
        # Classificateur
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16*5*5, 120), nn.ReLU(),
            nn.Linear(120, 84), nn.ReLU(),
            nn.Linear(84, 10)
        )
    def forward(self, x):
        return self.fc(self.conv2(self.conv1(x)))
```

Flux de données à travers LeNet :

Étape	Opération	Taille entrée	Taille sortie	Paramètres
Entrée	—	(B,1,28,28)	(B,1,28,28)	0
Conv1	Conv2d(1,6,5,pad=2)+ReLU	(B,1,28,28)	(B,6,28,28)	156
Pool1	MaxPool2d(2,2)	(B,6,28,28)	(B,6,14,14)	0
Conv2	Conv2d(6,16,5)+ReLU	(B,6,14,14)	(B,16,10,10)	2 416
Pool2	MaxPool2d(2,2)	(B,16,10,10)	(B,16,5,5)	0
Flatten	—	(B,16,5,5)	(B,400)	0
FC1	Linear(400,120)+ReLU	(B,400)	(B,120)	48 120
FC2	Linear(120,84)+ReLU	(B,120)	(B,84)	10 164
FC3	Linear(84,10)	(B,84)	(B,10)	850
TOTAL	—	—	—	~61 706

Ce qu'on dit à l'oral

LeNet alterne convolutions et poolings. Les couches convolutives extraient les features (bords, textures, formes). Le pooling réduit la dimension. Les couches fully-connected à la fin font la classification. Avec seulement 61k paramètres, LeNet atteint ~92% sur Fashion-MNIST, mieux que le MLP baseline qui a 6× plus de paramètres.

Section II.8 — Étude expérimentale — 6 configurations CNN

Qu'est-ce qu'on fait ici ?

On fait varier les hyperparamètres du CNN (padding, stride, type de pooling, nombre de filtres) pour mesurer leur impact sur l'accuracy.

Config	padding	stride	pooling	filtres C1/C2	Accuracy typique
1 (base)	2	1	max	6/16	~91%
2	0	1	max	6/16	~89% — sans padding, perte infos bords
3	2	2	max	6/16	~88% — stride=2 trop agressif
4	2	1	avg	6/16	~90% — avg-pool moins discriminant
5	2	1	max	16/32	~92% — plus de filtres, meilleure représentation
6	2	1	max	4/8	~88% — trop peu de filtres

Leçons de l'étude expérimentale :

- Le padding=2 avec noyau 5×5 est optimal — préserve l'info des bords
- Max-pooling > avg-pooling pour la classification d'images
- Plus de filtres améliore (jusqu'à un certain point — overfit sinon)
- stride=2 peut remplacer le pooling mais est moins stable

Ce qu'on dit à l'oral

L'étude expérimentale montre que la config de base (padding=2, max-pool, 6/16 filtres) est déjà bien optimisée. Doubler les filtres améliore légèrement. Supprimer le padding pénalise car les bords de l'image contiennent de l'information (ex. : la forme d'une chaussure commence au bord).

Section II.9 — Visualisation des cartes de caractéristiques

Qu'est-ce qu'on fait ici ?

On visualise ce que 'voit' chaque filtre après la première couche convolutive pour interpréter ce que le CNN a appris.

```
lenet.eval()
sample_img, label = test_fmnist[0]
x = sample_img.unsqueeze(0).to(device) # (1,1,28,28)

# Extraire les feature maps apres conv1
with torch.no_grad():
    feature_maps = lenet.conv1[0](x) # conv1[0] = la Conv2d uniquement
# shape : (1, 6, 28, 28) -> 6 cartes de 28x28

# Visualiser les 6 cartes
fig, axes = plt.subplots(1, 6, figsize=(12, 2))
for i, ax in enumerate(axes):
    ax.imshow(feature_maps[0, i].cpu(), cmap='viridis')
    ax.set_title(f'Filtre {i+1}')
```

Comment interpréter les feature maps ? Chaque carte montre où dans l'image le filtre correspondant s'est activé. Un filtre peut détecter les contours horizontaux (zones claires = contour présent), un autre les contours verticaux, un autre les textures, etc. Ces détecteurs sont **appris automatiquement** pendant l'entraînement.

Ce qu'on dit à l'oral

Les feature maps nous montrent ce que chaque filtre a appris à détecter. Sans jamais programmer "détecte les bords", le réseau apprend spontanément des filtres Gabor-like (détecteurs de contours orientés). C'est l'une des propriétés les plus remarquables des CNN : les représentations émergent naturellement de la rétropropagation.

Section II.10 — Question de synthèse — Partie II

Question : Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images, et quelles en sont les limites ?

Critère	MLP	CNN
Structure	Ignore la grille 2D	Exploite la localité spatiale
Paramètres	401k pour 784→512	61k pour LeNet complet
Invariance	Non invariant à la translation	Partiellement invariant (pooling)
Accuracy	~88%	~92%
Interprétabilité	Boîte noire totale	Feature maps visualisables

Limites du CNN :

- Pas invariant à la rotation (une chemise à 90° peut mal se classer)
- Pas invariant au changement d'échelle sans data augmentation
- Nécessite plus de données que le MLP pour éviter l'overfitting
- Architectures profondes (ResNet-50) nécessitent GPU et heures d'entraînement

PARTIE III — RNN, LSTM, GRU et Seq2Seq

Section III.1 — Fondements théoriques des modèles séquentiels

Qu'est-ce qu'on fait ici ?

On pose les bases mathématiques : modèle de langage, règle de la chaîne, perplexité. Ces concepts justifient l'utilisation des architectures récurrentes.

Modèle de langage — Règle de la chaîne

$$P(x_1, x_2, \dots, x_T) = P(x_1) \times P(x_2|x_1) \times P(x_3|x_1, x_2) \times \dots \times P(x_T|x_1, \dots, x_{T-1})$$

Un modèle de langage estime la probabilité d'une séquence de tokens. Il prédit chaque token à partir de tous les précédents.

Perplexité (PPL) : mesure à quel point le modèle est 'surpris' par les données de test. $PPL = \exp(\text{cross_entropy_loss})$. Un PPL de 100 signifie qu'en moyenne le modèle hésite entre 100 tokens à chaque position. **Plus bas = meilleur.**

```
# Calcul de la perplexité depuis la perte cross-entropie
ppl = math.exp(cross_entropy_loss)

# Exemple : perte=3.0 -> PPL=e^3.0 ≈ 20
# Le modèle hésite entre 20 mots possibles en moyenne
```

Pourquoi MLP et CNN échouent sur les séquences :

Architecture	Problème avec les séquences
MLP	Taille d'entrée fixe — impossible de traiter des séquences de longueur variable
CNN	Fenêtre locale fixe — contexte long ignoré, pas de mémoire temporelle
RNN	Traite séquences de longueur variable avec état caché h_t — solution naturelle

Ce qu'on dit à l'oral

Un modèle de langage prédit chaque mot à partir de tout le contexte précédent. La perplexité mesure la confiance du modèle : $PPL=20$ signifie qu'il hésite entre 20 mots à chaque étape. MLP et CNN ne peuvent pas traiter des séquences de longueur variable — le RNN a été conçu précisément pour ça.

Section III.2 — Corpus EN→FR et vocabulaire

Qu'est-ce qu'on fait ici ?

On construit un corpus de 80 paires anglais-français et une classe Vocabulaire qui gère la correspondance token↔index.

```
class Vocabulaire:
    def __init__(self):
        self.mot2idx = {'<PAD>':0, '<SOS>':1, '<EOS>':2, '<UNK>':3}
        self.idx2mot = {v:k for k,v in self.mot2idx.items()}

    def ajouter(self, mot):
        if mot not in self.mot2idx:
            idx = len(self.mot2idx)
            self.mot2idx[mot] = idx
            self.idx2mot[idx] = mot

    def encoder(self, phrase):
        return [self.mot2idx.get(m, 3) for m in phrase.lower().split()]
```

Token spécial	Index	Rôle
	0	Rembourrage pour uniformiser les longueurs de séquences
	1	Start Of Sentence — signal de début au décodeur
	2	End Of Sentence — signal d'arrêt de la génération
	3	Unknown — mots hors vocabulaire

Ce qu'on dit à l'oral

La classe Vocabulaire fait la correspondance entre les mots et les indices. Les tokens spéciaux sont essentiels : dit au décodeur de commencer, lui dit de s'arrêter, aligne les séquences en batch, gère les mots inconnus au test time.

Section III.3 — RNN simple — Modèle de langage

Qu'est-ce qu'on fait ici ?

On implémente un RNN simple pour la modélisation de langage : prédire le prochain token à partir du contexte.

Équation du RNN :

$$h_t = \tanh(W_h \times h_{t-1} + W_x \times x_t + b)$$

$$\hat{y}_t = \text{softmax}(W_y \times h_t + b_y)$$

h_t = état caché au temps t (mémoire du réseau)

x_t = embedding du token courant

$\hat{y}_t$ = distribution de probabilité sur le vocabulaire

```
class RNNLanguageModel(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=64, num_layers=1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.rnn = nn.RNN(embed_dim, hidden_size, num_layers,
                           batch_first=True, nonlinearity='tanh')
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, h=None):
        emb = self.embedding(x) # (B, T) -> (B, T, embed_dim)
        out, h_n = self.rnn(emb, h) # out: (B,T,hidden), h_n: (1,B,hidden)
        return self.fc(out), h_n # logits sur tout le vocabulaire
```

nn.Embedding : table de correspondance index→vecteur. Le mot d'index 5 dans un vocabulaire de 100 mots avec `embed_dim=32` donne un vecteur de 32 flottants. Ces vecteurs sont appris pendant l'entraînement.

Ce qu'on dit à l'oral

Le RNN maintient un état caché h_t qui se propage de t en t . À chaque étape, il prend le token courant (via embedding) et l'état précédent, et prédit le token suivant. Problème majeur : le gradient diminue exponentiellement en remontant dans le temps — le RNN oublie les contextes lointains.

Section III.4 — LSTM — Long Short-Term Memory

Qu'est-ce qu'on fait ici ?

Le LSTM résout le problème de gradient évanescent du RNN grâce à un mécanisme de portes qui contrôle ce qui est gardé ou oublié.

Porte	Formule (simplifiée)	Rôle
Oubli (f_t)	$\sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	Que doit-on oublier de la cellule ? (0=oublier, 1=garder)
Entrée (i_t)	$\sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	Quelle nouvelle info ajouter à la cellule ?
Cellule (c_t)	$f_t \cdot c_{t-1} + i_t \cdot \tanh(W_c \cdot \dots)$	État de la cellule — mémoire long terme
Sortie (o_t)	$\sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	Quelle partie de la cellule exposer comme h_t ?
h_t	$o_t \cdot \tanh(c_t)$	État caché — mémoire court terme

```
class LSTMLanguageModel(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=64, num_layers=1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_size, num_layers,
                             batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, hidden=None):
        emb = self.embedding(x)
        out, (h_n, c_n) = self.lstm(emb, hidden) # LSTM: 2 etats (h,c)
        return self.fc(out), (h_n, c_n)
```

Différence clé RNN vs LSTM :

RNN : 1 état caché h_t

LSTM : 2 états — h_t (court terme) ET c_t (long terme)

La cellule c_t peut transporter de l'information sur des centaines de steps sans que le gradient s'évanouisse.

Ce qu'on dit à l'oral

Le LSTM a 4 portes : oubli, entrée, cellule, sortie. La porte d'oubli décide ce qu'on supprime de la mémoire. La porte d'entrée décide ce qu'on ajoute. La cellule c_t est la mémoire long terme — elle peut rester intacte sur des dizaines de steps grâce à la porte d'oubli proche de 1.

Section III.5 — GRU — Gated Recurrent Unit

Qu'est-ce qu'on fait ici ?

Le GRU simplifie le LSTM en fusionnant les portes d'oubli et d'entrée en une seule porte de mise à jour, et en supprimant la cellule séparée.

Porte	Formule (simplifiée)	Rôle
Mise à jour (z_t)	$\sigma(W_z \cdot [h_{t-1}, x_t])$	Quelle proportion du passé vs présent garder ?
Réinitialisation (r_t)	$\sigma(W_r \cdot [h_{t-1}, x_t])$	Dans quelle mesure ignorer l'état précédent ?
h_t	$(1-z_t) \cdot h_{t-1} + z_t \cdot \tanh(W \cdot [r_t \cdot h_{t-1}, x_t])$	Mélange interpolé entre passé et présent

Critère	RNN	LSTM	GRU
Portes	0	3 (f, i, o)	2 (z, r)
États	1 (h_t)	2 (h_t, c_t)	1 (h_t)
Paramètres	Moins	Plus (×4 vs RNN)	Intermédiaire (×3 vs RNN)
Gradient évanesc.	Oui — grave	Non — résolu	Non — résolu
Vitesse	Plus rapide	Plus lent	Rapide comme RNN, stable comme LSTM
Séquences longues	Echec	Excellent	Très bon

Ce qu'on dit à l'oral

GRU est un LSTM allégé : 2 portes au lieu de 3, pas de cellule séparée. En pratique, GRU et LSTM donnent des résultats similaires, mais GRU est plus rapide à entraîner. Pour des séquences très longues ou des tâches complexes, LSTM reste préféré.

Section III.6 — BPTT et Gradient Clipping

Qu'est-ce qu'on fait ici ?

On explique la rétropropagation à travers le temps (BPTT) et on implémente le gradient clipping pour éviter l'explosion des gradients.

BPTT — Backpropagation Through Time

Pour un RNN sur une séquence de longueur T , le graphe de calcul s'étend sur T étapes. La rétropropagation remonte ce graphe de $t=T$ jusqu'à $t=1$. Le gradient est un produit de T matrices de poids $\rightarrow$ il peut :

- **Exploser** (produit de grandes valeurs) : gradients $\rightarrow \infty \rightarrow \text{NaN}$
- **S'évanouir** (produit de petites valeurs) : gradients $\rightarrow 0 \rightarrow$ rien n'apprend

Gradient Clipping — Solution à l'explosion

```
def train_lm(model, data, vocab_size, epochs=30, clip=1.0):
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    criterion = nn.CrossEntropyLoss(ignore_index=0) # ignore <PAD>

    for epoch in range(epochs):
        model.train()
        for X_b, y_b in data:
            optimizer.zero_grad()
            output, _ = model(X_b) # (B, T, vocab_size)
            loss = criterion(
                output.reshape(-1, vocab_size), # (B*T, V)
                y_b.reshape(-1) # (B*T,)
            )
            loss.backward()
            # GRADIENT CLIPPING : norme du gradient plafonnée à 1.0
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=clip)
            optimizer.step()
```

clip_grad_norm_(params, max_norm=1.0)

Si $\|\text{gradients}\| > 1.0 \rightarrow$ on les rescale pour que $\|\text{gradients}\| = 1.0$

Si $\|\text{gradients}\| \leq 1.0 \rightarrow$ on ne touche à rien

Cela évite les explosions sans empêcher l'apprentissage normal.

Ce qu'on dit à l'oral

La BPTT calcule les gradients en remontant le temps. Pour une séquence de 20 tokens, c'est un produit de 20 matrices. Si les valeurs sont >1 , le gradient explose. Si <1 , il s'évanouit. `clip_grad_norm_` plafonne la norme du gradient — c'est la solution standard pour l'explosion. Le LSTM/GRU résout l'évanouissement structurellement.

Section III.7 — Architecture Seq2Seq avec Encodeur-Décodeur

Qu'est-ce qu'on fait ici ?

On implémente l'architecture Seq2Seq pour la traduction EN→FR. L'encodeur lit la phrase source, le décodeur génère la traduction mot par mot avec Teacher Forcing.

```
class Encodeur(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=64):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_size, batch_first=True)

    def forward(self, x):
        emb = self.embedding(x)
        _, h_n = self.gru(emb) # h_n : vecteur de contexte
        return h_n # (1, B, hidden_size)

class Decodeur(nn.Module):
    def __init__(self, vocab_size, embed_dim=32, hidden_size=64):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, h):
        emb = self.embedding(x.unsqueeze(1)) # (B,) -> (B,1,E)
        out, h_n = self.gru(emb, h)
        return self.fc(out.squeeze(1)), h_n # logits + nouvel etat
```

Teacher Forcing :

```
# Pendant l'entraînement avec Teacher Forcing
for t in range(max_tgt_len):
    logit, h = decoder(tgt_input[:, t], h) # token reel comme entree
    loss += criterion(logit, tgt_out[:, t]) # pas le token predict

# Sans Teacher Forcing (inference) :
for t in range(max_len):
    logit, h = decoder(token_courant, h)
    token_courant = logit.argmax(dim=1) # le token predict devient l'entree
```

Teacher Forcing : pendant l'entraînement, on donne le vrai token de la traduction comme entrée du décodeur (même si la prédiction était fausse). Avantage : entraînement plus rapide et stable. Inconvénient : décalage train/inférence (exposure bias).

Ce qu'on dit à l'oral

L'encodeur lit toute la phrase anglaise et la compresse en un vecteur de contexte h_n . Le décodeur reçoit ce vecteur et génère la traduction française mot par mot. Teacher Forcing accélère l'entraînement en donnant le vrai token précédent au lieu du token prédit — comme si on donnait la réponse correcte à chaque étape.

Section III.8 — Décodage glouton (Greedy Decoding)

Qu'est-ce qu'on fait ici ?

On implémente le décodage glouton pour générer des traductions à l'inférence.

```
def decode_greedy(model, src_sentence, vocab_en, vocab_fr, max_len=12):
    model.eval()
    with torch.no_grad():
        # Encoder la phrase source
        src_idx = torch.tensor([vocab_en.encoder(src_sentence)]).to(device)
        h = model.encoder(src_idx)

        # Decoder mot par mot
        token = torch.tensor([vocab_fr.mot2idx['<SOS>']]).to(device)
        traduction = []
        for _ in range(max_len):
            logit, h = model.decoder(token, h)
            token = logit.argmax(dim=1) # choix du max
            mot = vocab_fr.idx2mot[token.item()]
            if mot == '<EOS>':
                break
            traduction.append(mot)
        return ' '.join(traduction)
```

Avantage : simple, rapide, $O(T)$ en temps. **Inconvénient** : sous-optimal — choisir le meilleur token à chaque étape ne garantit pas la meilleure séquence globale. Exemple : "le" peut être le meilleur token à $t=1$, mais mener à une mauvaise séquence si on avait choisi "la" à $t=1$.

Ce qu'on dit à l'oral

Le décodage glouton prend le token de probabilité maximale à chaque étape. C'est rapide mais myope : une bonne décision locale peut mener à une mauvaise séquence globale. Beam Search explore plusieurs hypothèses en parallèle pour trouver une meilleure séquence.

Section III.9 — Beam Search (k=3)

Qu'est-ce qu'on fait ici ?

Le Beam Search maintient k hypothèses (faisceaux) en parallèle et sélectionne la séquence avec le score global le plus élevé.

Algorithme Beam Search $k=3$:

- $t=0$: 3 meilleurs tokens $\rightarrow$ 3 hypothèses initiales avec leurs scores log
- $t=1$: chaque hypothèse génère V candidats $\rightarrow 3V$ candidats, garder les 3 meilleurs
- $t=2 \rightarrow t=\text{max_len}$: répéter — à chaque étape on explore $3 \times V$ candidats
- Fin : retourner l'hypothèse complète avec le score cumulé le plus élevé

```
def decode_beam(model, src_sentence, vocab_en, vocab_fr, beam_width=3, max_len=12):
    # Initialisation : liste de (score_log, [tokens], h)
    beams = [(0.0, [], h_init)]

    for _ in range(max_len):
        candidates = []
        for score, toks, h in beams:
            last_token = toks[-1] if toks else sos_idx
            logit, h_new = model.decoder(token_t, h)
            log_probs = F.log_softmax(logit, dim=1)
            # Top-k meilleurs tokens pour ce faisceau
            top_log_p, top_idx = log_probs.topk(beam_width)
            for lp, idx in zip(top_log_p[0], top_idx[0]):
                candidates.append((score + lp.item(),
                                   toks + [idx.item()], h_new))
            # Garder les beam_width meilleurs parmi tous les candidats
        beams = sorted(candidates, key=lambda x: x[0], reverse=True)[:beam_width]
```

Paramètre	k=1 (greedy)	k=3	k=10
Qualité	Sous-optimal	Bon compromis	Meilleur mais lent
Temps calcul	$O(T)$	$O(k \times T)$	$O(k \times T)$
Mémoire	Minimale	Modérée	$k \times$ la greedy

Ce qu'on dit à l'oral

Beam Search maintient $k=3$ hypothèses à chaque étape. À $t=1$ on a $3 \times V$ candidats, on garde les 3 meilleurs. Le score est la somme des log-probabilités (log pour éviter le underflow numérique). $k=3$ est un bon compromis : $3 \times$ plus lent que greedy mais traductions significativement meilleures.

Section III.10 — Score BLEU

Qu'est-ce qu'on fait ici ?

On implémente le score BLEU pour évaluer objectivement la qualité des traductions générées.

BLEU = Bilingual Evaluation Understudy

$$\text{BLEU} = \text{BP} \times \exp(\sum w_n \times \log(p_n))$$

p_n = précision des n -grammes entre la traduction candidate et la référence

BP = Brevity Penalty (pénalise les traductions trop courtes)

w_n = poids uniformes : $1/N$ (généralement $N=4$)

```
def bleu_score(candidate, reference, max_n=4):
    """candidate et reference sont des listes de mots"""
    c, r = len(candidate), len(reference)
    # Brevity Penalty
    bp = 1.0 if c >= r else math.exp(1 - r/c)

    log_avg = 0.0
    for n in range(1, max_n+1):
        # Compter les n-grammes communs
        ref_ngrams = Counter(zip(*[reference[i:] for i in range(n)]))
        cand_ngrams = Counter(zip(*[candidate[i:] for i in range(n)]))
        matches = sum((cand_ngrams & ref_ngrams).values())
        total = max(1, len(candidate) - n + 1)
        p_n = matches / total
        log_avg += (1/max_n) * math.log(p_n + 1e-10)
    return bp * math.exp(log_avg)
```

Score BLEU	Interprétation
< 0.10	Traduction de mauvaise qualité — peu de correspondance
0.10–0.30	Compréhensible mais avec des erreurs importantes
0.30–0.50	Bonne traduction par un humain non expert
> 0.50	Traduction de haute qualité (niveau systèmes commerciaux)

Limite du BLEU : il compare uniquement les n -grammes exacts. "Je suis content" et "Je suis heureux" sont sémantiquement identiques mais BLEU donne un score faible. Des métriques modernes comme ROUGE, BERTScore ou METEOR sont plus adaptées.

Ce qu'on dit à l'oral

BLEU mesure la proportion de n -grammes de la traduction produite qui apparaissent dans la traduction de référence. La Brevity Penalty pénalise les traductions courtes qui feraient un bon score en ne produisant que des mots sûrs. Sur notre petit corpus de 80 paires, un BLEU > 0.3 est un bon résultat.

Section III.11 — Question de synthèse — Partie III

Question : Dans quelle mesure les architectures récurrentes modélisent-elles efficacement les dépendances à long terme dans les séquences ?

Architecture	Dépendances courtes	Dépendances longues	Traduction
RNN simple	Oui	Non — gradient évanescent	Mauvaise
LSTM	Oui	Oui — cellule c_t	Bonne
GRU	Oui	Oui — porte z_t	Bonne
Transformer	Oui	Oui — attention globale $O(1)$	État de l'art

Limite fondamentale des RNN : le goulot d'étranglement

Dans le Seq2Seq basique, TOUTE l'information de la phrase source est compressée dans UN SEUL vecteur h_n de taille fixe (ex. 64 dimensions). Pour "The cat sat on the mat" → 64 flottants. Pour un paragraphe de 500 mots → toujours 64 flottants. Le mécanisme d'Attention (Bahdanau 2015 → Transformers) résout ce problème en permettant au décodeur de regarder TOUS les états de l'encodeur.

PARTIE ★ — Question transversale finale

Comment le deep learning adapte-t-il ses architectures à la structure des données ?

Critère	MLP	CNN	RNN/LSTM/GRU
Type de données	Tabulaire, vecteurs fixes	Images, volumes 3D	Séquences (texte, audio, séries)
Structure exploitée	Aucune — traitement global	Localité spatiale 2D	Ordre temporel
Biais inductif	Aucun	Invariance translation	Récurrence temporelle
Partage de poids	Non	Oui — même filtre sur image	Oui — même cellule à chaque pas
Input size	Fixe (n features)	Variable mais carré	Variable (T quelconque)
Paramètres	~10k–1M	~60k–100M	~10k–100M
Limite principale	Ignore structure	Non invariant rotation	Goulot d'étranglement (Seq2Seq)

Principe unificateur : chaque architecture encode a priori la structure typique de son domaine de données.

MLP : aucun a priori → flexible mais peu efficace

CNN : a priori de localité + invariance translation → optimal pour images

RNN : a priori de récurrence temporelle → optimal pour séquences

Transformer : a priori d'attention globale → optimal pour séquences longues

Section ★ — Concepts-clés à maîtriser absolument

Concept	Définition simple	Où on l'applique
nn.Module	Classe parente PyTorch — gère params, GPU, sauvegarde	Toutes les architectures
state_dict()	Dict des poids pour sauvegarder/recharger un modèle	I.7
Xavier init	Init poids selon fan_in+fan_out — évite vanishing/exploding	I.5
Data leakage	Utiliser des infos du futur ou de la cible en train	I.2 — stratify, split
corr2d manuelle	Glissement de noyau = ce que fait Conv2d	II.2
Gradient clipping	Plafonne la norme du gradient — évite explosion	III.6
Teacher Forcing	Donner le vrai token précédent au décodeur en train	III.7

Beam Search	k hypothèses parallèles — meilleur que greedy	III.9
BLEU	Précision des n-grammes entre candidat et référence	III.10
Perplexité	$PPL = \exp(CE_loss)$ — surprise du modèle de langage	III.1, III.6